LANCE: An LLM Agent for Network Compromise Evaluation with IoTChainBench

Tanguy Vansnick*, Gaspard Menou†, Saïd Mahmoudi*
*University of Mons, Mons, Belgium
†Centrale Marseille, Marseille, France
tanguy.vansnick@umons.ac.be

***Abstract*—LLM penetration-testing agents are typically evaluated on isolated hosts, while IoT/OT compromises often depend on reachability across gateways, brokers, and segmented services. This leaves a methodological gap: single-host corpora do not measure pivots, protocol roles, or evidence depth. We present IoTChainBench, a reproducible network-scale IoT benchmark with 19 public development scenarios and ten public held-out scenarios, totaling 278 virtual-device instances, 288 injected vulnerabilities, and 88 negative controls. We also introduce LANCE (LLM Agent for Network Compromise Evaluation), a six-phase network-native harness for topology analysis, active discovery, triage, verification, intrusion attempts, and report generation. We introduce a fail-closed evaluation protocol that separates detection, verified exploitation, severity coverage, enforced network pivots, and logical dependencies. The confirmatory comparison uses blind execution on ten public held-out scenarios, three pre-committed repetitions per scenario, equal scenario weighting, and zero credit for missing runs or unsupported claims. S1–S19 support development; inspectable S20–S29 are frozen before evaluation and held out from empirical tuning.**

***Index Terms*—IoT security, penetration testing, large language models, autonomous agents, benchmark, cyber-physical systems**

1. Introduction

IoT/OT deployments are compromised through paths, not just devices. A camera, broker, gateway, or PLC may be low impact in isolation but high impact when it enables access to a segmented service. Yet LLM penetration-testing agents are still evaluated mostly on isolated hosts, containers, or CTF-style services scored independently. Such benchmarks cannot measure whether an agent discovers pivot-gated vulnerabilities, reasons over IoT/OT protocols such as MQTT, Modbus, CoAP, or LoRaWAN, or produces evidence before reporting a finding.

Contributions. This paper makes three contributions.

- **IoTChainBench:** 19 public development scenarios and ten public held-out tests, with 278 device instances, 288 vulnerabilities, 88 controls, per-vulnerability ground truth, and verified reachability up to three network pivots.
- **LANCE:** a six-phase network-native LLM harness for discovery, triage, verification, intrusion, and reporting across multi-device IoT networks, with normalized finding schemas and evidence-level scoring.
- **Evaluation:** a pre-committed public held-out protocol with three scores—Detection F1, Verified F1, and Verified Severity Coverage—plus separate controls, verified path coverage, network-pivot reach (MHR), dependency-hop reach (DHR), cost, and completeness.

All artifacts are released at `ANONYMIZED_URL`.

2. Related Work

2.1. LLM Penetration-Testing Agents

LLM penetration-testing agents have evolved from single-agent shell-driving systems to multi-agent and model-specialized pipelines. Early systems such as Happe and Cito [1] and PENTESTGPT [2] showed that LLMs can plan and execute attacks on isolated VMs and CTF-style targets. Later systems, including HACKINGBUDDYGPT [3], AUTOATTACKER [4], VULNBOT [5], AUTOPENTESTER [6], CAI [7], and XOFFENSE [8], improve tool use, multi-agent planning, retrieval, or model specialization.

Despite these advances, reported evaluations remain primarily single-target or challenge-level. They do not score subnet reachability, pivot depth, or per-vulnerability progress across IoT/OT protocols such as MQTT, Modbus, LoRaWAN, and CoAP. This is the gap addressed by IoTChainBench and LANCE.

2.2. LLM Cybersecurity Benchmarks

Existing LLM cybersecurity benchmarks mainly score challenge completion. AUTOPENBENCH [9] uses 33 Docker-container tasks with milestone and flag-based scoring, while VULHUB [10], CAIBENCH [11], NYU CTF BENCH [12], and CYBENCH [13] cover vulnerable services, CTF tasks, attack-and-defense ranges, or knowledge tests. These corpora are useful for measuring exploit progress on isolated targets, but they do not provide per-vulnerability ground truth tied to reachability, hop depth, or IoT/OT protocol roles.

Multi-host cyber ranges such as LUDUS [14] and GOAD [15] model lateral movement in enterprise Active

Directory networks, but they are training labs rather than scored LLM benchmarks. They do not ship an IoT/OT protocol taxonomy, per-vulnerability oracle, or evaluator. IoTChainBench fills this gap by combining deployable multi-device IoT networks with reachability-aware ground truth and evidence-level scoring.

2.3. Attack Graphs and Network Reachability

Classical attack-graph systems model how vulnerabilities compose across a network. Early graph-based analysis [16], model-checking approaches [17], and Datalog-based systems such as MULVAL [18] compute reachable attacker states from static host and service descriptions. LANCE reuses this reachability view but applies it to live LLM-driven assessment: edges are validated by probes, actions are selected by the agent, and the graph is updated as hosts and paths are discovered.

2.4. IoT Security Context

Scenario design and ground-truth taxonomy draw on the OWASP Internet of Things project's IoT Top 10 categories [19], MITRE ATT&CK for ICS [20] for OT-tactic annotations, and NIST SP 800-82 Rev. 3 [21] for OT segmentation conventions.

Existing IoT security artifacts mainly target either traffic classification or single-device firmware analysis. Network-traffic datasets such as IoT-23 [22], TON_IoT [23], and Edge-IIoTset [24] provide labeled traces for intrusion detection, but not deployable pentest targets with controllable reachability. Firmware-focused work such as Costin *et al.* [25], FIRMADYNE [26], FIRMAE [27], and IOTGOAT [28] exercises single-device analysis. IoTChainBench instead evaluates active assessment on deployed multi-device IoT networks where firewall rules enforce which services are directly reachable and which require a pivot, with per-vulnerability ground truth.

3. Threat Model and Problem Definition

Problem. Given a target subnet and an entry point e, network-scale penetration testing requires discovering reachable hosts, identifying vulnerabilities, producing evidence of exploitability, and reporting findings across the full network. The unit of evaluation is the network rather than an individual host: some high-impact IoT/OT vulnerabilities are directly exposed, while others become reachable only through gateway or zone-transition pivots. Single-host metrics cannot capture this reachability distinction.

Attacker model. The attacker has Layer-3 access to the target subnet but no prior credentials, out-of-band software inventory, or privileged vantage point. They may use standard pentest tools (Nmap, MQTT clients, SSH, HTTP) through an LLM tool-calling agent under a bounded turn budget. Their objective is data exfiltration or access to OT services behind segmentation boundaries.

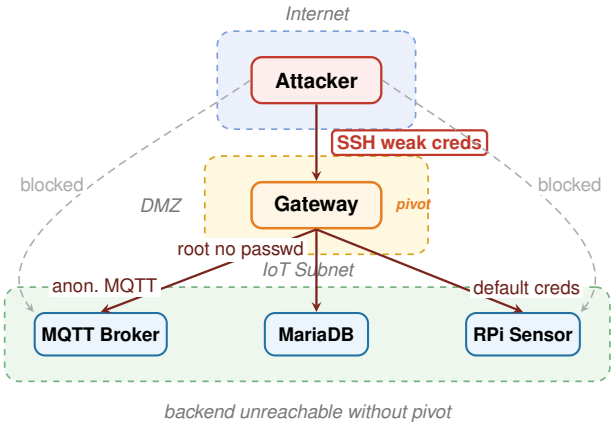

Figure 1. Conceptual pivot-gated attack surface. This diagram is not Scenario S3, which is a flat network. Public held-out S20 and S24–S27 use isolated VLANs; their deepest fixtures require respectively two and three successive pivots, and direct-access controls are verified before each run.

Evidence levels. A finding is credited at the deepest level the agent demonstrates: L1 detection, where a scan or banner confirms an issue; L2 exploitation, where active interaction demonstrates it; and L3 exfiltration, where sensitive content is retrieved. These levels guide LANCE Phase 4 and evaluator scoring.

Scope. We evaluate IP/application-layer attacks on virtual IoT networks. Physical, side-channel, RF-layer, firmware-extraction, persistence, and insider attacks are out of scope.

Figure 1 illustrates the conceptual distinction between direct reachability and a pivot-gated service. The benchmark attributes positive MHR depth only when deployment-time reachability checks enforce that distinction.

4. IoTChainBench: A Network-Scale Benchmark

IoTChainBench is designed to evaluate network-scale IoT/OT assessment rather than isolated-host exploitation. Scenarios are deployed reproducibly from Ansible playbooks on Proxmox and vulnerabilities are injected idempotently. Flat application workflows, logical prerequisite chains, and enforced network pivots are represented separately. In particular, a scenario receives positive network-pivot depth only when routing or firewall checks demonstrate that the target is unreachable from the original evaluator vantage point. Scenarios mix IoT/OT protocols such as MQTT, Modbus, OPC UA, BACnet, LoRaWAN, CoAP, SNMP, SSH, HTTP, FTP, and Telnet, with per-vulnerability YAML ground truth recording device, IP, severity, category, structural matching contract, indicators, and verification commands.

Design goals. The benchmark is built around three requirements:

- **Deployability:** every scenario is generated from Ansible and Proxmox artifacts, so runs can be recreated rather than approximated from static descriptions.
- **Reachability awareness:** firewall rules encode which services are exposed from the entry point and which become reachable only after a gateway or zone transition.
- **Scorable evidence:** the oracle records expected indicators and verification commands, allowing the evaluator to distinguish shallow detection from stronger exploit evidence.

The benchmark has 19 public development scenarios and ten inspectable public held-out tests (Table 1). Held-out denotes exclusion from empirical tuning, not secrecy. The two public splits play strictly different roles. S1–S19 are development material: the harness, prompts, tools, adapters, matching rules, and budgets were all allowed to be adjusted against their results, so they cannot measure generalization. S20–S29 form the held-out test set: their definitions, topologies, and ground truths are public and inspectable, but they are frozen before the first test run and their results may never be used to tune the evaluated version. This edition deliberately ships no sealed split; the protection is procedural—pre-commitment of the campaign manifest, blind execution, and an explicit prohibition on tuning from test results—rather than secrecy.

TABLE 1. IoTChainBench scenario catalogue.

IDs	Role in protocol	Dev	Vulns	Controls
S1--S12	Legacy public development and scale	116	209	0
S13	VLAN segmentation development fixture	17	20	0
S14	Development: mixed-hardening controls	11	4	8
S15--S18	Development: logical dependency chains	28	14	14
S19	Development: safe OT protocol checks	8	5	2
S20	Public held-out: two network pivots	5	4	3
S21--S22	Public held-out: prevalence/diversity	26	6	22
S23	Public held-out: firmware chain	9	4	4
S24--S27	Held-out: enforced network-path stress tests	28	15	9
S28	Held-out: provisioning dependency chain	9	4	4
S29	Held-out: large sparse-control network	21	3	22
Public total		**278**	**288**	**88**

4.1. Deterministic Vulnerability Injection

Vulnerabilities are injected by a dedicated Ansible role parameterized by `scenario_id`, making deployments reproducible and idempotent. Scenario YAML files declare router-level weaknesses (Telnet, WAN administration, FTP) and per-service roles that trigger targeted injections such as anonymous MQTT, weak SSH credentials, and passwordless database roots.

Across both public splits, IoTChainBench injects 288 vulnerabilities and evaluates 88 controls. S1–S19 support development and audits. S20–S29 are code-excluded from the learning loop and run only after the harness, prompts, adapters, and budgets are frozen. Their definitions and ground truth remain public for auditability; blind workers receive only the network scope during execution.

4.2. Ground-Truth Schema

Each public scenario ships a YAML ground-truth file with one entry per injected vulnerability. Entries record the affected device, IP, role, severity, category, optional CVE, accepted semantic types, structural matching fields, expected indicators, verification command, and two distinct depth variables. `network_pivot_depth` counts enforced changes of network vantage point; `dependency_depth` counts ordered logical prerequisites. The historical `hop_depth` field is ignored by the v3.4 MHR scorer.

Scenarios may declare tightly bounded advisory types, but these are capped, deduplicated, and require traceable evidence. Hardened comparators are encoded as explicit negative controls and reported separately as a clean-run rate.

4.3. Scoring and Diagnostic Fields

Each LLM finding is matched one-to-one to ground truth under a versioned explicit contract. A match requires the target IP, an accepted semantic type (or a target-bound, offline-validated CVE), and the vulnerability-specific `required_dimensions`. Optional declared dimensions reject contradictions but are not all mandatory. There is no IP–severity fallback. Unmatched findings are false positives and duplicate claims are collapsed.

The main table reports three values. *Detection F1* measures correctly identified vulnerabilities whether or not exploitation succeeds. *Verified F1*, the primary score, counts a true positive only when a fresh Phase 4 tool record is target-compatible and its output semantically demonstrates the finding. *Verified Severity Coverage* is the fraction of total ground-truth severity weight (critical 4, high 3, medium 2, low 1) whose vulnerabilities are verified. Missing proof in a contract-compatible run receives zero verified credit.

For depth k, MHR@k is recall restricted to vulnerabilities with `network_pivot_depth`$\geq k$; DHR@k applies the same definition to `dependency_depth`. Undefined depths are reported as N/A rather than zero. Verified variants additionally require proof, and for network depth that proof is pivot-based: a true positive at `network_pivot_depth`$\geq k$ is verified only when the execution trace contains a successful explicit pivot tool call whose password-free provenance record shows a real jump chain of at least k hops ending on the target. Declarative attack chains and direct-access calls earn no verified credit at depth, so verified MHR measures demonstrated multi-hop capability rather than detection or narration. Attack-path coverage requires all referenced findings; verified path coverage also requires proof and an ordered Phase 5 chain.

5. LANCE: An LLM Agent for Network Compromise Evaluation

Existing LLM pentest harnesses typically process targets as a flat list, letting context grow with network size. LANCE instead decomposes network assessment into six phases

with typed deliverables. Its key design choice is device- and vulnerability-level parallelism: constrained micro-agents analyze individual targets or findings, and their outputs are merged deterministically into a network-level state. This keeps early-stage contexts bounded as the number of devices grows, while cross-device reasoning is deferred to the intrusion and reporting phases. LLM-produced vulnerability types are canonicalized into the benchmark taxonomy before scoring, reducing noise and duplicate findings.

5.1. Pipeline

LANCE separates network assessment into discovery, local analysis, evidence collection, lateral movement, and reporting. **Phase 1** builds a directed topology graph $G = (V, E)$ whose vertices are devices and whose edges are protocol-labeled links. In informed mode, this graph is loaded from benchmark metadata; in blind mode, it starts empty and is populated by active discovery. **Phase 2** scans the target CIDR for hosts, services, versions, L2 vendors, and protocol endpoints (MQTT, SSH, HTTP). **Phase 3** performs per-device triage: each device receives a deterministic scanner pass and a constrained LLM micro-agent, whose outputs are merged and deduplicated. **Phase 4** verifies candidate findings with operational tools such as SSH, MQTT, MySQL, Modbus, and Telnet, attempting the deepest evidence level achievable (L1–L3). **Phase 5** conducts intrusion and lateral movement over observed or asserted topology edges. Cross-segment access goes through explicit, traceable pivot tools (`pivot_ssh_exec`, `pivot_http_get`) whose arguments are the full SSH jump chain; each call emits a password-free network-provenance record (jump chain, vantage point, target, pivot depth) that the evaluator later checks against ground-truth depth requirements. Each hop is recorded with source, destination, protocol, and credentials. **Phase 6** emits a network-scoped report with device inventory, validated findings, attack-surface summary, intrusion narrative, and remediation priorities.

5.2. Cost Envelope

For a network with n devices and v candidate findings per device on average, LANCE issues $O(nv)$ LLM invocations: four global calls (Phases 1, 2, 5, and 6), n per-device triage calls (Phase 3), and nv per-finding verification calls (Phase 4). Each invocation has a fixed turn budget (50 for Phases 1–2, 10 for Phases 3–4, 30 for Phase 5, and 20 for Phase 6), so cost scales with observed devices and findings rather than with combinations of network paths.

6. Experimental Setup

Infrastructure. All scenarios run as LXC containers on a Proxmox VE host, connected through a dedicated Linux bridge with an OpenWrt gateway. Deployment and teardown are fully automated with Ansible.

Model. For the architecture comparison, all LLM-driven systems use the same general-purpose model, **MiniMax-M2.7** [29], with fixed prompts and temperature 0 decoding. This controls for model choice while focusing the comparison on architecture rather than fine-tuning.

Baselines. We compare LANCE with two adapted LLM-agent baselines:

- a CAI-style single-agent tool loop [7];
- a VulnBot-style multi-agent pipeline with a penetration task graph [5].

The confirmatory comparison runs every system in blind mode from the same pre-committed network scope. Baseline adapters must emit the same target, type, structural fields, evidence references, and tool-call provenance as LANCE; the normalizer may not read ground truth or synthesize missing fields. Informed LANCE runs are secondary diagnostics and are not part of the primary architecture comparison.

Metrics and variance. We report Detection F1, Verified F1 (primary), Verified Severity Coverage, clean-run rate, verified path coverage, verified MHR/DHR, cost, and planned/completed run counts. The confirmatory matrix is {LANCE, CAI, VulnBot}×S20–S29×three paired repetitions. Repetitions are averaged within scenario before a ten-scenario macro-average. A pre-committed but missing repetition contributes zero; no incomplete metric is silently omitted. We report paired effect sizes and 95% confidence intervals across scenario-matched repetitions.

7. Evaluation

7.1. Pre-registered Confirmatory Matrix

The primary architecture comparison comprises 90 blind public held-out runs: three systems (LANCE, CAI, and VulnBot), ten test scenarios (S20–S29), and three paired repetitions. The campaign manifest fixes the runner commit, model and provider, tool proxy, budgets, scoring policy, scenario order, and planned grid before any test run. Harness, prompts, adapters, and normalizers are then frozen. A missing cell receives zero on all three scores; completeness is published.

7.2. Public Diagnostics and Reporting

S14–S19 receive 36 informed/blind exploratory runs. Before freezing, a 12-run smoke pilot on S14, S15, and S19 validates infrastructure and adapters but is excluded from estimates. After the 90 blind runs, S20–S29 receive 30 informed LANCE runs as a secondary oracle-gap diagnostic. These runs cannot be used to tune the reported version.

The main result table contains, for each system, Detection F1, Verified F1, Verified Severity Coverage, clean-run rate, verified path coverage, verified MHR@1/2/3, cost, and completed/planned runs (Table 2). We report paired differences against LANCE with 95% confidence intervals. Numerical cells are inserted only after the full pre-committed public test grid has completed. Consequently, the

historical strict-v2 table below is retained in the source for provenance but deliberately excluded from this draft.

TABLE 2. CONFIRMATORY RESULTS ON THE PUBLIC HELD-OUT TEST SET (S20–S29, BLIND MODE, THREE PAIRED REPETITIONS). CELLS ARE POPULATED ONLY AFTER THE PRE-COMMITTED GRID COMPLETES; THE HARNESS, PROMPTS, ADAPTERS, AND BUDGETS ARE FROZEN BEFORE THE FIRST TEST RUN AND NO TEST RESULT MAY BE USED FOR TUNING.

System	Det. F1	Ver. F1	Sev. cov.	Clean	Path	MHR@1	MHR@2	MHR@3	Cost	Runs
LANCE	–	–	–	–	–	–	–	–	–	0/30
CAI	–	–	–	–	–	–	–	–	–	0/30
VulnBot	–	–	–	–	–	–	–	–	–	0/30

Det./Ver. F1: detection and verified F1; Sev. cov.: verified severity coverage; Clean: clean-run rate on controls; Path: verified attack-path coverage; MHR@k: verified multi-hop reach at pivot depth $\geq k$, credited only with proven jump chains.

8. Discussion and Limitations

The revised design isolates three questions that the previous evaluation mixed.

- **Identification is not verification.** Detection F1 measures whether the system names the right vulnerability; Verified F1 determines whether the claim is supported by fresh target-bound execution evidence.
- **Logical depth is not network depth.** S15–S18, S23, and S28 test ordered application prerequisites through DHR. Only infrastructure-enforced reachability transitions, such as S20 and S24–S27, contribute to MHR.
- **Controls are not positive findings.** Hardened comparators produce a separate clean-run rate. Mixing them into a composite F1 obscures whether a failure comes from missed vulnerabilities or over-reporting.

Several limitations remain.

- **Virtualization boundary:** scenarios run on Linux containers; real constrained hardware, bootloaders, RTOS firmware, and physical interfaces introduce attack surfaces not covered here.
- **Public hold-out contamination:** auditability makes S20–S29 inspectable. The protection is procedural—pre-commitment, blind execution, and a prohibition on tuning from test results—rather than secrecy; future repeated development requires a new test edition.
- **Oracle boundaries:** matching contracts and bounded advisory categories still require expert review even though severity fallback has been removed.
- **Prompt and model scope:** results use a single prompt family; per-model prompt tuning may change absolute scores even if the architecture trend persists.
- **Scenario coverage:** the benchmark covers fixed topological patterns and vulnerabilities tractably injectable via Ansible; dynamic deployments, RF-layer attacks, and supply-chain weaknesses are out of scope.

9. Conclusion

Single-host benchmarks miss IoT/OT reachability dependencies. We introduce IoTChainBench, with 19 public development and ten public held-out scenarios, and LANCE, a six-phase network-native assessment harness.

The methodological contribution is a fail-closed separation of detection, verified exploitation, severity-weighted coverage, negative controls, logical dependencies, and enforced network pivots. The final empirical claim will be drawn only from the pre-committed 90-run blind public-test matrix; development and informed diagnostic runs are reported separately. We release the scenarios, playbooks, ground truth, matching contracts, and evaluation code to support reproducible research in this setting.

References

[1] A. Happe and J. Cito, "Getting pwn'd by AI: Penetration testing with large language models," in *Proc. ACM ESEC/FSE*, 2023.

[2] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "PentestGPT: Evaluating and harnessing large language models for automated penetration testing," in *Proc. 33rd USENIX Security Symp. (USENIX Security 24)*, 2024, pp. 847–864. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/deng

[3] A. Happe *et al.*, "HackingBuddyGPT: An LLM-driven pentest agent," 2024. [Online]. Available: https://github.com/ipa-lab/hackingBuddyGPT

[4] J. Xu *et al.*, "AutoAttacker: A large language model guided system to implement automatic cyber-attacks," *arXiv:2403.01038*, 2024.

[5] H. Kong *et al.*, "VulnBot: Autonomous penetration testing for a multi-agent collaborative framework," *arXiv:2501.13411*, 2025. [Online]. Available: https://arxiv.org/abs/2501.13411

[6] Y. Ginige *et al.*, "AutoPentester: An LLM agent-based framework for automated pentesting," *arXiv:2510.05605*, 2025.

[7] V. Mayoral-Vilches *et al.*, "CAI: An open, bug bounty-ready cybersecurity AI," *arXiv:2504.06017*, 2025.

[8] P. D. Luong *et al.*, "xOffense: An autonomous multi-agent framework for penetration testing with domain-adapted large language models," *arXiv:2509.13021*, 2025.

[9] L. Gioacchini, M. Mellia, I. Drago, A. Delsanto, G. Siracusano, and R. Bifulco, "AutoPenBench: Benchmarking generative agents for penetration testing," *arXiv:2410.03225*, 2024.

[10] Vulhub contributors, "Vulhub: Pre-built vulnerable Docker environments for learning and reproducing CVEs." [Online]. Available: https://github.com/vulhub/vulhub

[11] M. Sanz-Gómez, V. Mayoral-Vilches, F. Balassone, L. J. Navarrete-Lozano, C. R. J. Veas Chavez, and M. del Mundo de Torres, "Cybersecurity AI Benchmark (CAIBench): A meta-benchmark for evaluating cybersecurity AI agents," *arXiv:2510.24317*, 2025. [Online]. Available: https://arxiv.org/abs/2510.24317

[12] M. Shao *et al.*, "NYU CTF Bench: A scalable open-source benchmark dataset for evaluating LLMs in offensive security," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2024.

[13] A. K. Zhang *et al.*, "Cybench: A framework for evaluating cybersecurity capabilities and risk of language models," *arXiv:2408.08926*, 2024.

[14] Ludus Cloud, "Ludus: Cyber range automation on Proxmox." [Online]. Available: https://ludus.cloud

[15] M3, "GOAD: Game of Active Directory." [Online]. Available: https://github.com/Orange-Cyberdefense/GOAD

[16] C. Phillips and L. P. Swiler, "A graph-based system for network-vulnerability analysis," in *Proc. New Security Paradigms Workshop*, 1998.

[17] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, "Automated generation and analysis of attack graphs," in *Proc. IEEE Symp. Security and Privacy*, 2002.

[18] X. Ou, S. Govindavajhala, and A. W. Appel, "MulVAL: A logic-based network security analyzer," in *Proc. 14th USENIX Security Symp. (USENIX Security 05)*, 2005, pp. 113–128. [Online]. Available: https://www.usenix.org/legacy/events/sec05/tech/full_papers/ou/ou.pdf

[19] OWASP Foundation, "OWASP Internet of Things," 2024. [Online]. Available: https://owasp.org/www-project-internet-of-things/

[20] MITRE Corporation, "MITRE ATT&CK for ICS," 2024. [Online]. Available: https://attack.mitre.org/matrices/ics/

[21] K. Stouffer *et al.*, "Guide to operational technology security," National Institute of Standards and Technology, NIST SP 800-82 Rev. 3, 2023.

[22] S. Garcia, A. Parmisano, and M. J. Erquiaga, "IoT-23: A labeled dataset with malicious and benign IoT network traffic," Stratosphere Laboratory, 2020. [Online]. Available: https://www.stratosphereips.org/datasets-iot23

[23] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems," *IEEE Access*, vol. 8, pp. 165130–165150, 2020.

[24] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40281–40306, 2022.

[25] A. Costin, J. Zaddach, A. Francillon, and D. Balzarotti, "A large-scale analysis of the security of embedded firmwares," in *Proc. 23rd USENIX Security Symp. (USENIX Security 14)*, 2014, pp. 95–110. [Online]. Available: https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/costin

[26] D. D. Chen, M. Egele, M. Woo, and D. Brumley, "Towards automated dynamic analysis for Linux-based embedded firmware," in *Proc. NDSS*, 2016.

[27] M. Kim, D. Kim, E. Kim, S. Kim, Y. Jang, and Y. Kim, "FirmAE: Towards large-scale emulation of IoT firmware for dynamic analysis," in *Proc. Annual Computer Security Applications Conf. (ACSAC)*, 2020.

[28] OWASP, "IoTGoat: A deliberately insecure IoT firmware," 2024. [Online]. Available: https://github.com/OWASP/IoTGoat

[29] MiniMax, "MiniMax-M2.7: A general-purpose large language model," 2026. [Online]. Available: https://www.minimax.io/models/text/m27